Staff identity: one caller convention for every backend

Report of the work done on branch feat/gateway-api, 10 October 2026

Status	Done, committed and merged into `announce-jo-branch`. Not pushed.
Commit	`986373c6` on `feat/gateway-api`: 31 files, 689 lines added, 224 removed. The report is a second commit.
Scope	Close the admin edge header mismatch: every backend reads staff and members from `X-User-Id` + `X-User-Type`; order drops `X-Admin-Id`, messaging drops `X-Staff-Id`; the admin edge gains the staff chat routes and a WebSocket token filter.
Tests	223 tests in the six touched modules pass. No live manual run: see section 5.

1. The problem

- **Two languages between the admin edge and two backends.** The admin edge forwarded staff as `X-User-Id` with `X-User-Type=ADMIN`. The six staff routes of order read `X-Admin-Id` and the staff routes of messaging read `X-Staff-Id`. Nothing set those, so every staff action on order and messaging failed through the admin edge.
- **No staff chat through the admin edge.** After joining a support conversation, staff post into it through `/api/conversations/{id}/messages` and listen on `/ws/staff`. The admin edge routed neither path, denied everything under `/ws/**`, and could not carry a token on a WebSocket handshake.
- **Order's member routes did not check the caller type.** Seventeen handler parameters read `X-User-Id` alone, so a staff id would have been treated as the member with the same number.

You asked for the fix to follow the backend and to involve no outside party. It does: the backends keep the convention announce-management already had, the admin edge is completed around it, and nothing but our two edges and three backends is involved. Mapping headers per route at the edge was rejected: it is the `X-Seller-Id` pattern removed on 1 October.

2. What was built

The convention, in the framework

The `security` package of `cqrs-reactive` gained a `Caller` record (id, type MEMBER or ADMIN, roles) and a `Callers` helper that every service uses:

Call	Who passes	Otherwise
`Callers.member(headers)`	type MEMBER	401 `no-principal` without an id, 403 `not-a-member` for staff
`Callers.staff(headers)`	type ADMIN	401 `no-principal`, 403 `not-staff` for a member
`Callers.any(headers)`	either	401 `no-principal`
`Callers.memberIfAny(headers)`	a member, else empty	never fails for a visitor or staff

An id with a missing or unknown type is 403 `unknown-caller-type`; a non-numeric id is 400. `X-Seller-Id`, `X-Staff-Id` and `X-Admin-Id` are read nowhere any more.

Per module

Module	Change
cqrs-reactive	`Caller`, `Callers`, five unit tests; section 5a of its guide.

Module	Change
order	The six staff routes read `Callers.staff`; the seventeen member parameters read `Callers.member`. `X-Admin-Id` gone. The end-to-end test sends members and staff as the edges do.
messaging	`Principals` is a thin layer over `Callers`; the admin controller and both WebSocket handshakes read the same pair. `X-Staff-Id` gone. Tests and the load driver updated.
announce-management	Its `Callers` delegates to the framework. No behaviour change.
gateway-admin	Routes `/api/conversations/**` and `/ws/**` to messaging; `WebSocketTokenFilter` turns `?access_token=` into the bearer header; `ROLE_ADMIN` on `/ws/**`; `X-Staff-Id` and `X-Admin-Id` stripped from clients; the dev profile points order at 8083 and the WebSocket route at messaging.
Docs	`doc/frontend-edges.md` section 11 and the admin edge section, `doc/cqrs-reactive.md` section 5a, the order and messaging context docs, the messaging architecture diagram, CLAUDE.md (one build fact).

3. Before the work: the merge

`feat/messaging` held five clean, green, unmerged commits that touched the same messaging files. It was merged into this branch first (`832055d0`, no conflicts), so `announce-jo-branch` now carries the messaging race fixes and their report as well.

4. Verification

Automated tests

One Maven run over cqrs-reactive, cqrs-reactive-test, announce-management, order and messaging, then messaging again on its own (section 5 says why), plus a run of gateway-admin:

Module	Tests	Result
cqrs-reactive	59	pass
cqrs-reactive-test	15	pass
announce-management	60	pass, one skipped (a listing-import test not touched by this work)
order	33	pass
messaging	33	pass
gateway-admin	23	pass
Total	223	no failures

What the new and changed tests prove:

- **order.** A member on a staff route gets 403 `not-staff`, staff on a member route 403 `not-a-member`, nobody 401, and `X-Admin-Id` alone is not an identity.
- **messaging.** Opening a conversation needs a member (401 without, 403 for staff), the support queue needs staff (401 without, 403 for a member), `X-Staff-Id` alone is refused, staff read and post with the pair, and the staff WebSocket handshake carries the pair.
- **gateway-admin.** The staff identity reaches order and messaging with a verifiable service token; spoofed `X-Staff-Id` and `X-Admin-Id` are dropped; the `/ws/staff` handshake with the query token is forwarded with the identity and without the JWT; a member token is refused on the staff chat routes.

5. What was not done, and what got in the way

- **No live manual run.** The plan ended with a run through the admin edge against a local messaging service. Another session was running messaging on port 8086 from its worktree against the shared EventSourcingDB, Kafka and MySQL. A second node would have joined its consumer groups and disturbed that session's work, so the manual check was not done. The automated coverage runs the real messaging service on containers for the backend part and the real edge against a WebSocket-capable stand-in for the edge part.
- **The same instance stalled the messaging suite once.** `MessagingEndToEndTest` keeps its read model in the live database and resets the three checkpoint rows at start. While the other instance ran, it kept writing its own checkpoints (event id 13581, from the long-lived EventSourcingDB) into those rows, so the suite's projections, fed by a fresh container, never caught up: six timeouts. The suite passed as soon as the instance stopped. This is the likely cause of the single unexplained failure noted on 1 October as well. Run that suite when no messaging instance uses `announce_reactive_db`.
- **Admin screens.** The admin SPA's phase-2 screens run on mocks that call list endpoints the order backend does not have (`GET /api/admin/auctions`, `GET /api/admin/orders`, `GET /api/admin/payouts`). Wiring those screens to the real backends is separate work.
- **Roles inside the backends.** `X-User-Roles` is forwarded and parsed but not checked; the edge enforces `ROLE_ADMIN` and `ROLE_SUPER` on its own routes.
- **Listings admin routes.** `/api/admin/listings/**` in announce-management still has no caller check of its own.

6. State of the branches

Branch	State
`feat/gateway-api`	code commit `986373c6` plus this report; clean
`announce-jo-branch`	fast-forwarded to the same commits; not pushed
`feat/messaging`	fully contained in announce-jo-branch since this work

Nothing to clean this time: no configuration file changed shape. Design record: `doc/frontend-edges.md` section 11.